

UNIVERSIDADE PAULISTA – UNIP EaD

Projeto Integrado Multidisciplinar

**Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas**

Yasmine Lima Santana - 2526592

Geovanna Félix Monteiro - 2521699

Luan Lopes Bezerra - 2503460

João Pedro Carneiro da Silva - 2540975

Vitória Freitas - 2523279

Sofia de Castro Ongaro - 2528018

**Plataforma para pesquisa clínica e manufatura
farmacêutica**

Santana de Parnaíba, SP

2025

Yasmine Lima Santana - 2526592

Geovanna Félix Monteiro - 2521699

Luan Lopes Bezerra - 2503460

João Pedro Carneiro da Silva - 2540975

Vitória Freitas - 2523279

Sofia de Castro Ongaro - 2528018

Plataforma para pesquisa clínica e manufatura farmacêutica

Projeto Integrado Multidisciplinar em
Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do
título de tecnólogo em Análise e Desenvolvimento
de Sistemas, apresentado à Universidade Paulista
– UNIP EaD.

Orientador (a): Tarcísio Peres

Santana de Parnaíba, SP

2025

aqui vai o resumo

abstract

sumário

introducao

1. Redes de Computadores e Sistemas Distribuídos

A plataforma distribuída proposta para integrar pesquisa clínica e manufatura farmacêutica depende fortemente de uma infraestrutura de rede estável, resiliente e preparada para lidar tanto com dados em tempo real quanto com requisitos rígidos de auditoria. Por isso, o desenho da arquitetura de rede foi pensado para garantir **baixa latência, isolamento de tráfego sensível, segurança regulatória e continuidade operacional mesmo diante de falhas.**

1.1 Topologia da Rede e Arquitetura Distribuída

Será desenvolvido um protótipo de infraestrutura lógica baseado em uma **topologia hierárquica**, modelo amplamente utilizado em ambientes corporativos e industriais por oferecer organização clara dos fluxos, facilidade de escalabilidade e controle eficiente de tráfego.

Essa topologia é formada por três camadas principais:

- **Núcleo (Core):** responsável pelo roteamento de alto desempenho e pela interligação entre todos os módulos críticos da plataforma, como o sistema de pesquisa clínica, o ambiente de manufatura e os bancos de dados distribuídos.
- **Distribuição:** camada intermediária que aplica políticas de segurança, controle de QoS e redireciona o tráfego de forma otimizada.
- **Acesso:** onde ficam os dispositivos finais, como sensores laboratoriais, estações de análise, máquinas da linha de produção e servidores locais.

Esse modelo garante que os dados sensíveis, como informações clínicas ou registros industriais auditáveis trafeguem de maneira organizada, mantendo performance e segurança mesmo em cenários de carga elevada.

1.2 Segmentação e Segurança: Uso de VLANs

Para garantir isolamento adequado entre os diferentes fluxos de informação, a rede será dividida em **VLANs (Virtual Local Area Networks)**.

Essa segmentação é essencial em ambientes farmacêuticos, pois:

- Separa dados clínicos de informações de manufatura;
- Isola tráfego administrativo, de pesquisa, de IA e dos sensores fabris;

- Reduz a superfície de ataque e facilita auditorias de conformidade (como FDA 21 CFR Part 11);
- Evita congestionamento, melhorando o desempenho da plataforma como um todo.

Cada VLAN terá políticas próprias de QoS e níveis de prioridade diferentes. Por exemplo, os pacotes que carregam sinais de sensores laboratoriais, que exigem latência mínima, serão priorizados em relação a fluxos menos críticos.

1.3 Roteamento Redundante com OSPF e BGP

Como a plataforma precisa funcionar mesmo quando partes da infraestrutura falham, o projeto incluirá a configuração de **protocolos de roteamento redundantes**, como **OSPF (Open Shortest Path First)** ou **BGP (Border Gateway Protocol)**.

Estes protocolos permitem que a rede:

- Descubra automaticamente caminhos alternativos;
- Se recupere rapidamente em casos de quebra de links ou falhas de switches/roteadores;
- Mantenha a comunicação entre os módulos da plataforma sem interrupção.

Na prática, isso significa que, mesmo com uma pane parcial (por exemplo, uma queda de um link entre a área clínica e o ambiente fabril), os dados continuarão fluindo por rotas alternativas, preservando a continuidade das análises e dos processos.

1.4 Rede Definida por Software (SDN) com OpenDaylight

Para centralizar e flexibilizar o controle da rede, o protótipo utilizará uma solução de **SDN (Software Defined Networking)**, aplicada por meio de um controlador como o **OpenDaylight**.

Essa abordagem separa o plano de controle do plano de dados, permitindo:

- Automatizar políticas de segurança;
- Responder rapidamente a mudanças no tráfego ou em cenários de falha;

- Aplicar regras dinâmicas de QoS para fluxos críticos;
- Monitorar performance em tempo real.

O SDN é especialmente útil em ambientes regulados, pois permite registrar e auditar todas as decisões de roteamento, facilitando a conformidade com GLP, GCP e GMP.

1.5 Tolerância a Falhas e Continuidade Operacional

O objetivo da arquitetura distribuída proposta é garantir que a plataforma continue funcionando mesmo diante de falhas parciais.

Para validar isso, o protótipo será avaliado com base em métricas como:

- **Latência:** tempo que os dados levam para atravessar a rede;
- **Throughput:** volume de dados transmitidos em determinado período;
- **Disponibilidade:** percentual de tempo em que a plataforma fica acessível.

Durante os testes, serão simulados cenários de pane, como desligamento de switches ou saturação de links, para demonstrar como a topologia hierárquica, o roteamento redundante e as políticas do SDN conseguem manter a operação sem interrupções perceptíveis.

1.6 Integração com a Plataforma e o Ecossistema Digital

A rede distribuída criada não funciona isoladamente, ela é parte essencial do ecossistema digital que integra pesquisa clínica, manufatura e modelos de Inteligência Artificial explicáveis.

Por meio dessa infraestrutura:

- Sensores laboratoriais enviam dados em tempo real para os módulos de análise;
- Os algoritmos de IA recebem informações atualizadas com baixa latência;
- Os dados críticos trafegam de forma isolada e compatível com auditorias regulatórias;

- Equipes de pesquisa e manufatura têm acesso contínuo às informações, garantindo velocidade e precisão na tomada de decisões.

aqui vai ia

3. Programação Estruturada em C

Na disciplina de Programação Estruturada em C, desenvolvemos uma biblioteca nativa que contém rotinas de filtragem digital, compressão adaptativa de registros biomédicos e gravação segura em arquivos binários, no total foram 12 funções criadas para cumprir tais funcionalidades.

Criamos funções para comparar sinais entre pacientes e dispositivos, limpeza de sinais biomédicos, economizar espaço e transmissão, eliminar flutuações aleatórias, armazenamento eficiente de sinais contínuos, reduzir logs estáveis de sensores, análise de complexidade de sinais, verificação de integridade e auditoria e gravação/leitura de imagens.

Também adicionamos integração por FFI (Foreign Function Interface), utilizando a linguagem Python, onde executamos as funções que foram criadas na biblioteca de C nesse arquivo Python usando a biblioteca ctypes.

3.1 Arquivos criados

Arquivo “bib.h”, para declarar para o compilador quais funções e structs existem no módulo:

```
// bib.h

#ifndef BIB_H
#define BIB_H

// struct para os coeficientes
typedef struct
{
    float b0, b1, b2;
    float a1, a2;
} Coeficientes;

// struct para as imagens
typedef struct
```

```
{
    char imagem_nome[50];

    char imagem_formato[10];

    unsigned long tamanho;

    unsigned char *imagem_dados;
} Imagens;

void sinais_normalizados(int vetor[], int tamanho, int
valor_maximo_absoluto, float vetor_saida[]);

void filtroFir(int amostras_entrada[], float coeficientes[], int
tamanho_entrada, int tamanho_coeficientes, float sinal_filtrado[]);

void filtroIIR(float entrada[], float saida[], int tamanho,
Coeficientes coeficientes);

void downsample(float entrada[], int tamanho, int fator, float
saida[]);

void denoising(float entrada[], int tamanho, int janela, float
saida[]);

void codificacao_delta(float entrada[], int tamanho, float saida[]);
void decodificacao_delta(float entrada[], int tamanho, float saida[]);

void RLEAdaptativo(float entrada[], int tamanho, float threshold,
float valores[],int contagens[], int *tamanho_saida);

void entropia_janela(float entrada[], int tamanho, int janela, float
saida[]);

float integridade_checksum(float entrada[], int tamanho);

void gravar_imagem(const Imagens *imagem, const char *nome_arquivo);
Imagens* ler_imagem_binaria(const char *nome_arquivo);

#endif
```

Arquivo “bib.c”, onde foi criado as funções da biblioteca, antes de cada função adicionamos um comentário para explicar resumidamente o que ela irá fazer:

```
//bib.c

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "bib.h"
#include <string.h>

// formula valor_normalizado = valor_original / valor_maximo_absoluto
// Normalizar significa pegar esse vetor e ajustar todos os valores
para caber em uma faixa fixa
// Pré-processamentoz

void sinais_normalizados(int vetor[], int tamanho, int
valor_maximo_absoluto, float vetor_saida[]){

    if (valor_maximo_absoluto == 0) {

        printf("Erro: valor_maximo_absoluto não pode ser zero.\n");

        return;

    }

    int i;

    for(i = 0; i < tamanho; i++){

        float valor = (float)vetor[i] / valor_maximo_absoluto;

        vetor_saida[i] = valor;

    }
```

```

}

// Filtro FIR

/* FIR = Finite Impulse Response. Pega as últimas N amostras e faz
média ponderada delas. */

void filtroFir(int amostras_entrada[], float coeficientes[], int
tamanho_entrada, int tamanho_coeficientes, float sinal_filtrado[]){

    float acumulador = 0.0f;

    int i, k;

    for(i = 0; i < tamanho_entrada; i++){

        acumulador = 0;

        for(k = 0; k < tamanho_coeficientes; k++){

            if(i - k >= 0) {

                acumulador += amostras_entrada[i - k] *
coeficientes[k];

            }

        }

        sinal_filtrado[i] = acumulador;

    }

}

//Filtro IIR(biquad)

```

```

/* Um filtro IIR (Infinite Impulse Response) usa realimentação
(feedback), ele utiliza amostras atuais e anteriores da entrada e
amostras anteriores da saída.

Formula:  $y[n] = b_0 * x[n] + b_1 * x[n-1] + b_2 * x[n-2] - a_1 * y[n-1] - a_2 * y[n-2]$ 

*/

void filtroIIR(float entrada[], float saida[], int tamanho,
Coeficientes coeficientes){

    float x1 = 0.0f;

    float x2 = 0.0f;

    float y1 = 0.0f;

    float y2 = 0.0f;

    int i;

    for(i = 0; i < tamanho; i++){

        float x0 = entrada[i];

        float y0 = (coeficientes.b0 * x0) + (coeficientes.b1 * x1) +
(coeficientes.b2 * x2)- (coeficientes.a1 * y1) - (coeficientes.a2 *
y2);

        saida[i] = y0;

        x2 = x1;

        x1 = x0;

        y2 = y1;

        y1 = y0;

    }

}

/*Downsampling: significa diminuir a taxa de amostragem do sinal.*/
void downsample(float entrada[], int tamanho, int fator, float

```

```
saida[]){

    int indice_saida = 0;

    int i;

    for(i = 0; i < tamanho; i++) {

        if(i % fator == 0){

            saida[indice_saida] = entrada[i];

            indice_saida++;

        }

    }

}

/*Denoising: reduzir o ruído do sinal suavizando variações bruscas,
mas mantendo a forma do sinal (picos e tendências).*/

void denoising(float entrada[], int tamanho, int janela, float
saida[]){

    int metade = janela / 2;

    int i, k;

    for(i = 0; i < tamanho; i++){

        float acumulador = 0;

        int contador = 0;

        for(k = -metade; k <= metade; k++) {

            int indice = i + k;

            if(indice < 0 || indice >= tamanho){

                continue;

            }

        }

    }

}
```

```

        acumulador += entrada[indice];

        contador++;

    }

    saida[i] = acumulador / contador;

}

}

// Rotinas compressão adaptativa
/*
Delta Encoding

A codificação delta é uma técnica de compressão simples onde não
armazenamos os valores originais do sinal, mas sim as diferenças entre
uma amostra e a anterior.

*/

void codificacao_delta(float entrada[], int tamanho, float saida[]){

    // Primeiro valor é copiado sem alteração

    saida[0] = entrada[0];

    int i;

    // Calcula a diferença das próximas amostras

    for(i = 1; i < tamanho; i++){

        saida[i] = entrada[i] - entrada[i - 1];

    }

}

```

```

/*
decodificação delta

A função recebe um vetor com valores codificados por delta
(diferenças) e reconstrói o sinal original somando cada diferença ao
valor acumulado anterior.

*/

void decodificacao_delta(float entrada[], int tamanho, float saida[]){

    saida[0] = entrada[0];

    int i;

    for(i = 1; i < tamanho; i++){

        saida[i] = saida[i-1] + entrada[i];

    }

}

/*
RLE Adaptativo

O RLE adaptativo é uma variação do RLE tradicional.

No RLE clássico, comprime sequências de valores iguais consecutivos
(ex.: 5 5 5 5 → (5,4)).

No RLE adaptativo, você define um limiar (threshold) para considerar
valores "suficientemente próximos", e não apenas idênticos.

*/

void RLEAdaptativo(float entrada[], int tamanho, float threshold,
float valores[], int contagens[], int *tamanho_saida) {

    float valor_atual = entrada[0];

    int contador = 1;

```

```

int indice_saida = 0;

for (int i = 1; i < tamanho; i++) {
    if (fabs(entrada[i] - valor_atual) <= threshold) {
        contador++;
    } else {
        valores[indice_saida] = valor_atual;
        contagens[indice_saida] = contador;
        indice_saida++;

        valor_atual = entrada[i];
        contador = 1;
    }
}

// Salva o último grupo
valores[indice_saida] = valor_atual;
contagens[indice_saida] = contador;

*tamanho_saida = indice_saida + 1;
}

//Entropia de Janela (Window Entropy)
/*
A entropia mede o nível de imprevisibilidade (caos) do sinal dentro de
uma janela (um pedaço do vetor).

Em sinais biomédicos ou industriais, isso serve para:
detectar momentos de instabilidade,
diferenciar atividade (sinal variando muito) e repouso (sinal

```

```

estável),

identificar eventos anormais.

- Quanto maior a entropia, mais variáveis e imprevisíveis são os
valores naquela janela.

- Quanto menor a entropia, mais repetitivos e previsíveis são os
valores.*/

void entropia_janela(float entrada[], int tamanho, int janela, float
saida[]){

    int i, k;

    for(i = 0; i <= tamanho - janela; i++){

        float freq[100] = {0};

        for(k = 0; k < janela; k++){

            int indice = (int) entrada[i + k];

            freq[indice]++;

        }

        float entropia = 0.0f;

        for(k = 0; k < 100; k++){

            if (freq[k] > 0) {

                float prob = (float)freq[k] / janela;

                entropia += prob * log2(prob);

            }

        }

    }

```

```

        // -entropia, pois resultado da entropia fica positivo (como
deve ser)

        saida[i] = -entropia;

    }
}

/*
Integridade (Checksum / Verificação de Integridade)

Quando transmitimos ou armazenamos dados (em dispositivos médicos,
pesquisa clínica ou indústria farmacêutica), precisamos garantir que o
sinal não foi alterado ou corrompido.

A técnica mais simples e bastante usada é o Checksum:

Um checksum calcula um valor numérico (soma ou operação matemática)
baseado no conteúdo do vetor.

Se os dados forem modificados, o checksum muda – permitindo
identificar erro.
*/

float integridade_checksum(float entrada[], int tamanho){

    float soma = 0.0f;

    int i;

    for(i = 0; i < tamanho; i++){

        soma = soma + entrada[i];

    }

    return soma;

}

// Gravar a imagem

void gravar_imagem(const Imagens *imagem, const char *nome_arquivo) {

```

```

FILE *arquivo = fopen(nome_arquivo, "wb");

if (!arquivo) {

    printf("Erro ao abrir arquivo.\n");

    return;

}

fwrite(imagem->imagem_nome, sizeof(char),
sizeof(imagem->imagem_nome), arquivo);

fwrite(imagem->imagem_formato, sizeof(char),
sizeof(imagem->imagem_formato), arquivo);

fwrite(&imagem->tamanho, sizeof(unsigned long), 1, arquivo);

fwrite(imagem->imagem_dados, sizeof(unsigned char),
imagem->tamanho, arquivo);

fclose(arquivo);

printf("Imagem '%s' salva em '%s'\n", imagem->imagem_nome,
nome_arquivo);
}

// Ler a imagem criada
Imagens* ler_imagem_binaria(const char *nome_arquivo) {

FILE *arquivo = fopen(nome_arquivo, "rb");

if (!arquivo) {

    printf("Erro ao abrir o arquivo para leitura.\n");

    return NULL;
}

```

```
}

// Aloca a estrutura dinamicamente

Imagens *imagem = (Imagens *)malloc(sizeof(Imagens));

if (!imagem) {

    printf("Erro ao alocar memória para a imagem.\n");

    fclose(arquivo);

    return NULL;

}

// Lê os metadados (nome, formato e tamanho)

fread(imagem->imagem_nome, sizeof(char),
sizeof(imagem->imagem_nome), arquivo);

fread(imagem->imagem_formato, sizeof(char),
sizeof(imagem->imagem_formato), arquivo);

fread(&imagem->tamanho, sizeof(unsigned long), 1, arquivo);

// Aloca memória para os bytes da imagem

imagem->imagem_dados = (unsigned char *)malloc(imagem->tamanho);

if (!imagem->imagem_dados) {

    printf("Erro ao alocar memória para os dados da imagem.\n");

    free(imagem);

    fclose(arquivo);

    return NULL;

}

// Lê os bytes reais da imagem

fread(imagem->imagem_dados, sizeof(unsigned char),
imagem->tamanho, arquivo);
```

```

fclose(arquivo);

printf("Imagem '%s.%s' carregada com sucesso (%lu bytes)\n",
       imagem->imagem_nome, imagem->imagem_formato,
       imagem->tamanho);

return imagem; // retorna a struct alocada dinamicamente
}

```

Arquivo “main.c”, onde executamos as funções criada na biblioteca em um arquivo C:

```

// arquivo main.c
#include <stdio.h>
#include <stdlib.h>
#include "bib.h"
#include <string.h>

int main(){

    // Rotinas Filtragem Digital

    int vetor[7] = {-32700, -15000, -8000, 0, 12000, 25000, 31000};

    float vetor_saida[7];

    int i;

    //pré-processamento

    sinais_normalizados(vetor, 7, 32767, vetor_saida);

    printf("Resultado do pre-processamento\n");
}

```

```
for(i = 0; i < 7; i++){  
    printf("%.1f\n", vetor_saida[i]);  
}  
  
// Filtro FIR  
  
int amostras_entrada[4] = {4, 8, 6, 5};  
float coeficientes[3] = {0.33, 0.33, 0.33};  
float sinal_filtrado[4];  
  
filtroFir(amostras_entrada, coeficientes, 4, 3, sinal_filtrado);  
  
printf("Resultado do Filtro FIR\n");  
  
for(i = 0; i < 4; i++){  
    printf("%.1f\n", sinal_filtrado[i]);  
}  
  
// Filtro IIR  
  
Coeficientes coeficiente;  
  
coeficiente.b0 = 0.2929;  
coeficiente.b1 = 0.5858;  
coeficiente.b2 = 0.2929;  
coeficiente.a1 = -0.0000;  
coeficiente.a2 = 0.1716;
```

```
float entrada[8] = { 0.0, 0.5, 0.8, 0.3, -0.2, -0.5, -0.3, 0.1 };  
float saida[8];  
  
filtroIIR(entrada, saida, 8, coeficiente);  
  
printf("Resultado do Filtro IIR\n");  
  
for(i = 0; i < 8; i++){  
    printf("%.1f\n", saida[i]);  
}  
  
// Downsampling  
float entrada_downsample[4] = {1.3, 2.0, 4.3, 3.1};  
int tamanho_entrada = 4;  
int fator = 2;  
int tamanho_saida = (tamanho_entrada + fator - 1) / fator;  
float saida_downsample[tamanho_saida];  
  
downsample(entrada_downsample, tamanho_entrada, fator,  
saida_downsample);  
  
printf("Resultado do Downsampling\n");  
  
for(i = 0; i < tamanho_saida; i++){  
    printf("%.1f\n", saida_downsample[i]);  
}  
  
// Denoising
```

```
float entrada_denoising[4] = {4.3, 3.0, 1.3, 2.1};

int tamanho_denoising = 4;

int janela = 4;

float saida_denoising[tamanho_denoising];

denoising(entrada_denoising, tamanho_denoising, janela,
saida_denoising);

printf("Resultado do Denoising\n");

for(i = 0; i < tamanho_denoising; i++){
    printf("%.1f\n", saida_denoising[i]);
}

//Rotinas Compressão Adaptativa
// Codificação delta

float entrada_delta[4] = {4.3, 5.0, 4.6, 4.8};

float saida_delta[4];

codificacao_delta(entrada_delta, 4, saida_delta);

printf("Resultado do Codificacao delta\n");

for(i = 0; i < 4; i++){
    printf("%.1f\n", saida_delta[i]);
}

// decodificação delta
```

```

float saida_decodificada[4];

decodificacao_delta(saida_delta, 4, saida_decodificada);

printf("Resultado do decodificacao delta\n");

for(i = 0; i < 4; i++){
    printf("%.1f\n", saida_decodificada[i]);
}

// RLE Adaptativo

float entradaRLE[4] = {2.3, 4.0, 3.1, 7.8};

float threshold = 2;

float valores[4];

int contagens[4];

int tamanho_saidaRLE;

RLEAdaptativo(entradaRLE, 4, threshold, valores, contagens,
&tamanho_saidaRLE);

printf("Resultado do RLE Adaptativo\n");

for(i = 0; i < tamanho_saidaRLE; i++){
    printf("Valor: %.1f | Contagem: %d\n", valores[i],
contagens[i]);
}

//entropia da janela

// Entropia da janela

```

```
int tamanho_entropia = 4;

float entrada_entropia[4] = {1.3, 4.0, 2.1, 5.8};

int janela_entropia = 3;

// tamanho da saída = tamanho da entrada - janela + 1
int tamanho_saida_entropia = tamanho_entropia - janela_entropia +
1;

float saida_entropia[tamanho_saida_entropia];

entropia_janela(entrada_entropia, tamanho_entropia,
janela_entropia, saida_entropia);

printf("Resultado da Entropia da Janela\n");
for(i = 0; i < tamanho_saida_entropia; i++){
    printf("%.1f\n", saida_entropia[i]);
}

//Verificação de Integridade
float entrada_integridade[4] = {1.2, 3.0, 4.5, 2.3};

float checksum = integridade_checksum(entrada_integridade, 4);

printf("Resultado da Verificação de Integridade (Checksum):
%.2f\n", checksum);

unsigned char dados_para_imagem[] = { 0xFF, 0xD8, 0xA0, 0x3F,
0x7C, 0x42 }; // simulação

unsigned long tamanho = sizeof(dados_para_imagem);
```

```
Imagens Imagem;

strcpy(Imagem.imagem_nome, "amostra_clinica");

strcpy(Imagem.imagem_formato, "jpg");

Imagem.tamanho = tamanho;

Imagem.imagem_dados = dados_para_imagem;

gravar_imagem(&Imagem, "amostra_clinica.bin");

// lendo imagens

Imagens *img_lida = ler_imagem_binaria("amostra_clinica.bin");

if (img_lida != NULL) {

    printf("Nome: %s\n", img_lida->imagem_nome);

    printf("Formato: %s\n", img_lida->imagem_formato);

    printf("Tamanho: %lu bytes\n", img_lida->tamanho);

    // exemplo de uso: acessar o primeiro byte

    printf("Primeiro byte: 0x%X\n", img_lida->imagem_dados[0]);

}

// quando terminar, liberar memória:

free(img_lida->imagem_dados);

free(img_lida);

return 0;

}
```

Arquivo “python.py”, onde executamos as funções em um arquivo Python:

```
# arquivo python

import ctypes
import os

from ctypes import c_float, c_int, c_ulong, c_char, POINTER, Structure
from ctypes import *

# Caminho absoluto da DLL (garante que o Python a encontre)
os.chdir(os.path.dirname(__file__))

dll_path = os.path.abspath(os.path.join(os.path.dirname(__file__),
"bib.dll"))

lib = ctypes.CDLL(dll_path)

# Mapeando as structs do C
class Imagens(Structure):
    _fields_ = [
        ("imagem_nome", c_char * 50),
        ("imagem_formato", c_char * 10),
        ("tamanho", c_ulong),
        ("imagem_dados", POINTER(ctypes.c_ubyte))
    ]

class Coeficientes(Structure):
    _fields_ = [
        ("b0", c_float),
        ("b1", c_float),
        ("b2", c_float),
```

```

        ("a1", c_float),

        ("a2", c_float),

    ]

# Configurando os protótipos das funções

lib.sinais_normalizados.argtypes = (POINTER(c_int), c_int, c_int,
POINTER(c_float))

lib.sinais_normalizados.restype = None

lib.filtroIIR.argtypes = (POINTER(c_float), POINTER(c_float), c_int,
Coeficientes)

lib.filtroIIR.restype = None

lib.filtroFir.argtypes = [POINTER(c_int), POINTER(c_float), c_int,
c_int, POINTER(c_float)]

lib.filtroFir.restype = None

# Entrada agora como INT, igual ao C
filtroFirEntrada = (c_int * 4)(4, 8, 6, 5)
coeficientesFir = (c_float * 3)(0.33, 0.33, 0.33)
filtroFirSaida = (c_float * 4)()

lib.filtroFir(filtroFirEntrada, coeficientesFir, 4, 3, filtroFirSaida)
print("=====")

print("Resultado da Filtro Fir:")

for x in filtroFirSaida:

    print(round(x, 2))

```

```

entrada = (c_int * 7)(-32700, -15000, -8000, 0, 12000, 25000, 31000)
saida = (c_float * 7)()

lib.sinais_normalizados(entrada, 7, 32767, saida)

print("=====")
print("Resultado da normalização:")
for x in saida:
    print(round(x, 3))
print("=====")

# Chamando filtro IIR
coef = Coeficientes(0.2929, 0.5858, 0.2929, -0.0000, 0.1716)

entrada_iir = (c_float * 8)(0.0, 0.5, 0.8, 0.3, -0.2, -0.5, -0.3, 0.1)
saida_iir = (c_float * 8)()

lib.filtroIIR(entrada_iir, saida_iir, 8, coef)

print("Resultado do filtro IIR:")
for x in saida_iir:
    print(round(x, 3))
print("=====")

#####

# --- Tipos auxiliares ---
c_float_p = POINTER(c_float)

```

```
c_int_p = POINTER(c_int)

# MAPEAMENTO DAS FUNÇÕES DA DLL

# downsample
lib.downsample.argtypes = [c_float_p, c_int, c_int, c_float_p]
lib.downsample.restype = None

# denoising
lib.denoising.argtypes = [c_float_p, c_int, c_int, c_float_p]
lib.denoising.restype = None

# codificação delta
lib.codificacao_delta.argtypes = [c_float_p, c_int, c_float_p]
lib.codificacao_delta.restype = None

# decodificação delta
lib.decodificacao_delta.argtypes = [c_float_p, c_int, c_float_p]
lib.decodificacao_delta.restype = None

# RLE adaptativo
lib.RLEAdaptativo.argtypes = [c_float_p, c_int, c_float, c_float_p,
c_int_p, POINTER(c_int)]
lib.RLEAdaptativo.restype = None

# entropia janela
lib.entropia_janela.argtypes = [c_float_p, c_int, c_int, c_float_p]
lib.entropia_janela.restype = None
```

```

# checksum

lib.integridade_checksum.argtypes = [c_float_p, c_int]
lib.integridade_checksum.restype = c_float

#chamar essas funções no Python

entrada = (c_float * 4)(1.3, 2.0, 4.3, 3.1)
saida = (c_float * 2)()

lib.downsample(entrada, 4, 2, saida)

print("Downsample:", list(saida))

entrada = (c_float * 4)(4.3, 5.0, 4.6, 4.8)
cod = (c_float * 4)()
decod = (c_float * 4)()
janela_entropia = 2
entropia = (c_float * (4 - janela_entropia + 1))()
denoising = (c_float * 4)()

lib.codificacao_delta(entrada, 4, cod)
lib.decodificacao_delta(cod, 4, decod)

lib.entropia_janela(entrada, 4, janela_entropia, entropia)
lib.denoising(entrada, 4, 2, denoising)

print("=====")

print("Delta encoder: ", list(cod))

```

```

print("=====")
print("Delta decoded: ", list(decoded))
print("=====")
print("Entropia: ", list(entropia))
print("=====")
print("Denoising: ", list(denoising))
print("=====")

entrada = (c_float * 4)(2.3, 4.0, 3.1, 7.8)
valores = (c_float * 4)()
contagens = (c_int * 4)()
tam_saida = c_int()

lib.RLEAdaptativo(entrada, 4, 2.0, valores, contagens,
byref(tam_saida))

print("RLE:")
for i in range(tam_saida.value):
    print(f"Valor: {valores[i]} | Contagem: {contagens[i]}")
print("=====")

entrada = (c_float * 4)(1.2, 3.0, 4.5, 2.3)

checksum = lib.integridade_checksum(entrada, 4)

print("Checksum:", checksum)
print("=====")

```

```

# Gravar imagem:
lib.gravar_imagem.argtypes = (POINTER(Imagens), ctypes.c_char_p)
lib.gravar_imagem.restype = None

# criar objeto de imagem
img = Imagens()
img.imagem_nome = b"amostra_clinica"
img.imagem_formato = b"jpg"
img.tamanho = 6

# exemplo de bytes da imagem
dados = (ctypes.c_ubyte * img.tamanho)(10, 20, 30, 40, 50, 60)
img.imagem_dados = dados

# salvar usando a DLL
lib.gravar_imagem(ctypes.byref(img), b"amostra_clinica.bin")

print("Imagem salva com sucesso!")
print("=====")

lib.ler_imagem_binaria.argtypes = (ctypes.c_char_p,)
lib.ler_imagem_binaria.restype = ctypes.POINTER(Imagens)

# Ler a imagem salva
img_ptr = lib.ler_imagem_binaria(b"amostra_clinica.bin")

if img_ptr:
    img = img_ptr.contents

```

```
print("\nImagem lida:")

print("Nome:", img.imagem_nome.decode())

print("Formato:", img.imagem_formato.decode())

print("Tamanho:", img.tamanho)

print("=====")
```

3.2 Comandos utilizados para a execução do código no terminal do Visual Studio Code

Utilizamos a IDE Visual Studio Code para criação dos arquivos de código do projeto, utilizando o compilador GCC 64 bits.

Comandos para compartilhar as funções entre os arquivos C: “gcc -o app main.c bib.c”, comando utilizado para executar o arquivo no “main.c”: “./app.c”.

Comando para compartilhar as funções para usar no arquivo de Python: gcc -shared -o bib.dll bib.c "-Wl,--out-implib,bib.a".

3.3 Resposta no Terminal do arquivo “main.c”:

```
PROBLEMAS  SAIDA  CONSOLE DE DEPURACAO  TERMINAL  PORTAS
● PS C:\Users\vitor\OneDrive\Documentos\faculdade-repositorio\faculdade-
Resultado do pre-processamento
-1.0
-0.5
-0.2
0.0
0.4
0.8
0.9
Resultado do Filtro FIR
1.3
4.0
5.9
6.3
Resultado do Filtro IIR
0.0
0.1
0.5
0.7
0.3
-0.3
-0.5
-0.2
Resultado do Downsampling
1.3
4.3
Resultado do Denoising
2.9
2.7
2.7
2.1
Resultado do Codificacao delta
4.3
0.7
-0.4
0.2
Resultado do decodificacao delta
4.3
5.0
4.6
4.8
Resultado do RLE Adaptativo
Valor: 2.3 | Contagem: 3
Valor: 7.8 | Contagem: 1
Resultado da Entropia da Janela
1.6
1.6
Resultado do Checksum: 11.00
Imagem 'amostra_clinica' salva em 'amostra_clinica.bin'
Imagem 'amostra_clinica.jpg' carregada com sucesso (6 bytes)
Nome: amostra_clinica
Formato: jpg
Tamanho: 6 bytes
Primeiro byte: 0xFF
```

3.4 Resposta no Terminal do arquivo “python.py”:

```

C:\Users\vitor\OneDrive\Documentos\faculdade-repositorio\faculdade-repositorio\segundo_semestre\Python\Python312\python.exe c:/Users/vitor/OneDrive/Documentos/faculdade-repositorio/faculdade-repositorio/python.py
Resultado da Filtro Fir:
1.32
3.96
5.94
6.27
=====
Resultado da normalização:
-0.998
-0.458
-0.244
0.0
0.366
0.763
0.946
=====
Resultado do filtro IIR:
0.0
0.146
0.527
0.678
0.261
-0.292
-0.484
-0.243
=====
Downsample: [1.2999999523162842, 4.300000190734863]
=====
Delta encoder: [4.300000190734863, 0.6999998092651367, -0.40000009536743164, 0.20000028610229492]
=====
Delta decoded: [4.300000190734863, 5.0, 4.599999904632568, 4.800000190734863]
=====
Entropia: [1.0, 1.0, -0.0]
=====
Denoising: [4.650000095367432, 4.633333206176758, 4.800000190734863, 4.699999809265137]
=====
RLE:
Valor: 2.299999952316284 | Contagem: 3
Valor: 7.800000190734863 | Contagem: 1
=====
Checksum: 11.0
=====
Imagem 'amostra_clinica' salva em 'amostra_clinica.bin'
Imagem salva com sucesso!
=====
Imagem 'amostra_clinica.jpg' carregada com sucesso (6 bytes)

Imagem lida:
Nome: amostra_clinica
Formato: jpg
Tamanho: 6
=====

```

4. Análise e Projeto de Sistemas

4.1 Padrão de projeto escolhido para plataforma

Para o desenvolvimento da plataforma distribuída para gestão integrada de pesquisa clínica e manufatura farmacêutica, escolhemos o padrão de projeto API gateway. Antes de explicar o que é esse padrão e seus benefícios, em primeiro lugar iremos explicar o que é um padrão de projeto.

Padrões de projetos são soluções para problemas comuns no desenvolvimento de um software, é um conceito geral que pode ser implementado para resolver um problema recorrente dentro do seu código, podendo ser customizado para cumprir tal função.

A API gateway é um padrão de projeto que funciona como ponto de entrada entre diferentes tipos de sistemas, simplificando e otimizando a comunicação entre esses sistemas, assim sendo ótima para plataformas distribuídas.

Dentro dela existem diferentes padrões de design, para o desenvolvimento do nosso sistema escolhemos o Two-tier gateway, onde trabalha com dois gateways, o primeiro irá atuar como um proxy reverso roteando solicitações para o segundo gateway, esse segundo roteia para serviços do back-end, separando assim gateways para o lado do cliente e um para lado do servidor (back-end). Esse padrão de design possibilita a utilização de diferentes tipos de hardware ou gateways para cada camada, promovendo escalabilidade e segurança para aplicação.

Existem vários benefícios na utilização da API gateway, vamos citar alguns deles. Um dos benefícios é a segurança, pois ela fornece mecanismos de autenticação e autorização para o controle de acesso às APIs de back-end, incluindo mecanismo de criptografia e controle de tráfego. Possui também o monitoramento e registros de logs fornecidos pela API gateway para todas as API do sistema registradas. Os gateways de API distribuem de maneira eficiente a carga de trabalho, promovendo escalabilidade e disponibilidade dos serviços. Concluimos que se nossa plataforma cumprir tais funcionalidades conseguiremos ficar de acordo com a GAMP 5 e FDA 21 CFR Part 11, em relação a integridade e segurança dos dados.

4.2 Requisitos funcionais e não funcionais do sistema

Tabela 01 - requisitos funcionais da plataforma

Módulo	ID	Descrição	Justificativa
Gestão de acesso	RF-001	O sistema deve implementar MFA (Multifatorial) para os usuários	Assim o sistema ficará de acordo com a FDA 21 CFR Part 11
Controle de acesso	RF-002	O sistema deve gerenciar os perfis dos usuários com base em roles (RBAC)	Com isso o sistema terá mais boas práticas, em relação fabricação (BPF), ficando de acordo com o GAMP 5
Pesquisa clínica	RF-003	O sistema deve permitir assinatura eletrônica e deve possuir formulário de consentimento.	Necessário para cumprimento da FDA 21 CFR Part 11
Farmacovigilância	RF-004	O sistema deve permitir registro e rastreamento de eventos adversos	Com isso teremos uma melhor gestão para segurança clínica
Manufatura Farmacêutica	RF-005	O sistema deve ter integrado sistema de controle de lotes, com a	Essencial para rastreabilidade de lote

		função de rastrear a produção de medicamentos para centros de pesquisa.	
--	--	---	--

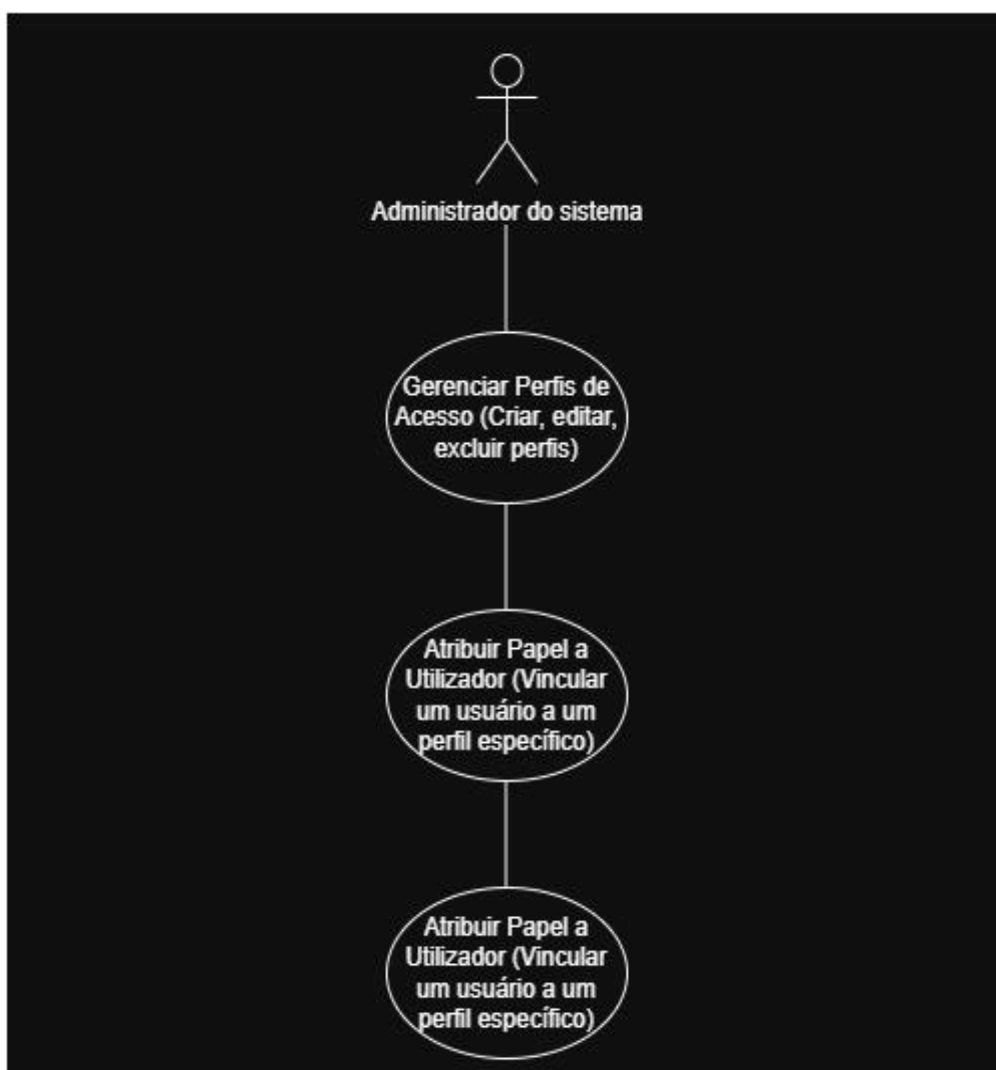
Tabela 02 - requisitos não funcionais da plataforma

Módulo	ID	Descrição	Justificativa
Desempenho e Escalabilidade	RNF-001	Baixa Latência: O tempo de resposta do sistema deve ser até 2 segundos	Importante para experiência do usuário e eficiência operacional
Disponibilidade do sistema	RNF-002	Disponibilidade: O sistema de estar disponível 99.9% do tempo para os componentes essenciais.	Para continuidade dos negócios em pesquisas clínicas e manufatura.
Confiabilidade e Segurança	RNF-003	Segurança: Dados sensíveis devem ser criptografados em repouso e em trânsito	Fundamental para o sistema estar de acordo com a LGPD ou GDPR
Usabilidade e Operação	RNF-004	A interface deve ser intuitiva	Assim, podemos ter uma redução de erros e facilitar o uso da

			plataforma para o usuário
Arquitetura	RNF-005	O sistema deve ser capaz de se comunicar e trocar dados com outros sistemas	Vital para um ecossistema digital integrado

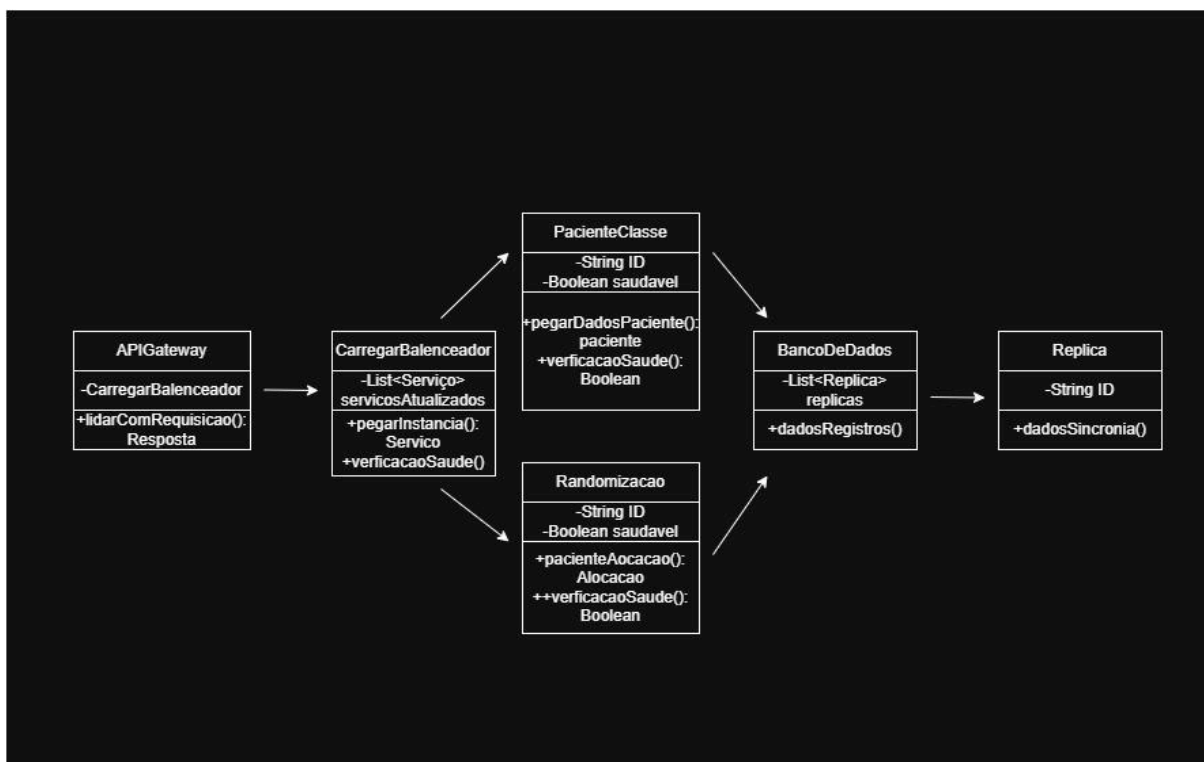
4.3 Diagramas produzidos com a ferramenta CASE - Draw.io

Diagrama de Casos de Uso (Baseado no RF-002):



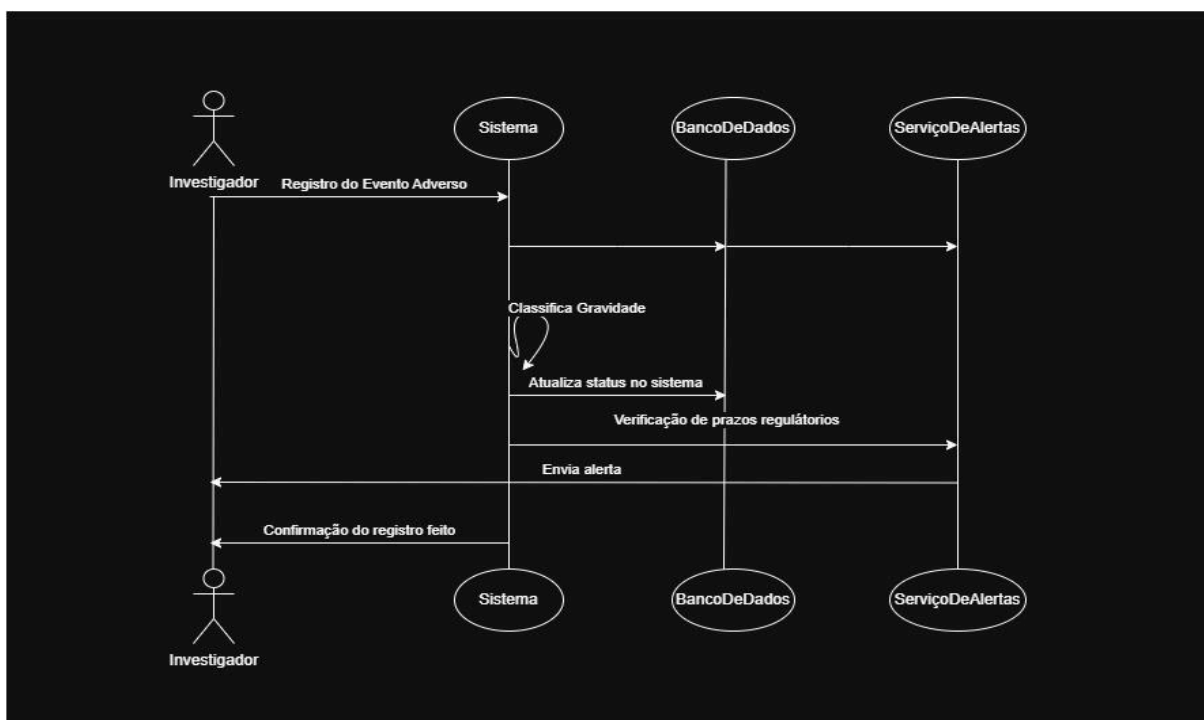
fonte: Autoria própria

Diagrama de Classes (Baseado no RNF-002):



fonte: Autoria própria

Diagrama de Sequência (Baseado no RF-004):



fonte: Autoria própria

Aqui vai a conclusão

referências

Fontes de análise de sistema

<https://refactoring.guru/pt-br/design-patterns/what-is-pattern>

<https://blog.engdb.com.br/api-gateway-2/>

https://cdn.novusautomation.com/downloads/superview_21cfrpart11_pt.pdf

<https://www.estrategiaconcursos.com.br/blog/requisitos-funcionais-nao-funcionais-cef-ti/>

Bibliografia

- CISCO. **Enterprise Campus Design**. Disponível em: <https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html> - Acesso: Novembro de 2025 .
- MOY, J. T. **OSPF Version 2. RFC 2328**, 1998. Disponível em: <https://www.rfc-editor.org/info/rfc2328> - Acesso: Novembro de 2025.
- REKHTER, Y. et al. **A Border Gateway Protocol 4 (BGP-4). RFC 4271**, 2006. Disponível em: <https://www.rfc-editor.org/info/rfc4271> - Acesso: Novembro de 2025.
- FOOD AND DRUG ADMINISTRATION (FDA). **Part 11 Electronic Records; Electronic Signatures — Scope and Application**. 21 CFR Part 11. Disponível em: <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/part-11-electronic-records-electronic-signatures-scope-and-application> - Acesso: Novembro de 2025.
- OpenDaylight – Documentação Oficial. Disponível em: <https://www.opendaylight.org/about> - Acesso: Novembro de 2025.